Module - 3:
Clean Code &
Software Craftsmanship

- Amjad Hossain
18th Feb, 2026

Why Clean Code Matters in Financial Systems

- High Maintainability and Stability
- Reduced Technical Debt
- Improved Collaboration and Onboarding
- Security and Risk Mitigation
- Preventing Financial Loss due to Bugs
- Security and Vulnerability Reduction

Clean Code Principles: Why??

- Readability
- Maintainability
- Simplicity
- Easy to understand
- Testability
- Easy to modify to adapt changes

Clean Code Principles **VS** Design Patterns

Feature	Design Principles	Design Patterns
Focus	**Why** (Philosophy, Rules of Thumb)	**How** (Implementation, Solutions)

Clean Code Principles **VS** Design Patterns

Feature	Design Principles	Design Patterns
Focus	**Why** (Philosophy, Rules of Thumb)	**How** (Implementation, Solutions)
Usage	Applied early (architecture/design)	Applied during coding/refactoring

Clean Code Principles **VS** Design Patterns

Feature	Design Principles	Design Patterns
Focus	**Why** (Philosophy, Rules of Thumb)	**How** (Implementation, Solutions)
Usage	Applied early (architecture/design)	Applied during coding/refactoring
Nature	Abstract, Theoretical, High-level	Concrete, Practical, Low-level
Scope	Broad, applicable to entire systems	Localized, specific to a problem

Clean Code Principles **VS** Design Patterns

Feature	Design Principles	Design Patterns
Focus	**Why** (Philosophy, Rules of Thumb)	**How** (Implementation, Solutions)
Usage	Applied early (architecture/design)	Applied during coding/refactoring
Nature	Abstract, Theoretical, High-level	Concrete, Practical, Low-level
Scope	Broad, applicable to entire systems	Localized, specific to a problem
Examples	SOLID, DRY, KISS, YAGNI	Singleton, Factory, Observer, Strategy

Clean Code Principles

- **Meaningful names:** Use descriptive, pronounceable names for variables, functions, and classes that reveal their purpose.
  - Verbs for functions (CalculateTotal)
  - Nouns for Classes (UserAccount)
  - Noun if variable represent object or data (user, total, etc.)
  - Plural Nouns for arrays or lists (users, cars etc.)
  - For boolean variable use prefix like: is, has, can

Clean Code Principles

- ➔ Meaningful names
- ➔ **Small Functions:** **Functions should be small, focused, and ideally take few arguments.**

Clean Code Principles

- ➔ Meaningful names
- ➔ Small Functions
- ➔ **Boy Scout Rule:** **Always leave the campground cleaner than you found it – improve code quality with every change.**

Clean Code Principles

- ➔ Meaningful names
- ➔ Small Functions
- ➔ Boy Scout Rule
- ➔ **Avoid Magic number/string:** **Replace hard-coded literals with named constants for better context.**

Clean Code Principles

- ➔ Meaningful names
- ➔ Small Functions
- ➔ Boy Scout Rule
- ➔ Avoid Magic number/string
- ➔ **Consistent Formatting:** **Adhere to a unified style, such as using appropriate indentation, spacing, and casing (e.g., camelCase vs. snake_case)**

Clean Code Principles

- ➔ Meaningful names
- ➔ Small Functions
- ➔ Boy Scout Rule
- ➔ Avoid Magic number/string
- ➔ Consistent Formatting
- ➔ **RIP (Return If Possible):** **In functions, return wherever possible. It reduces nested logic**

Clean Code Principles

- ➔ Meaningful names
- ➔ Small Functions
- ➔ Boy Scout Rule
- ➔ Avoid Magic number/string
- ➔ Consistent Formatting
- ➔ **RIP (Return If Possible):** **In functions, return wherever possible. It reduces nested logic**

```
public Boolean IsValidEmail(String email) {
    if (email != null) {
        if (!email.isEmpty()) {
            if (email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$")) {
                return true;
            }
        }
    }
    return false;
}
```

Clean Code Principles

- ➔ Meaningful names
- ➔ Small Functions
- ➔ Boy Scout Rule
- ➔ Avoid Magic number/string
- ➔ Consistent Formatting
- ➔ **RIP (Return If Possible):** **In functions, return wherever possible. It reduces nested logic**

```
public Boolean IsValidEmail(String email) {
    if ( email == null )return false;
    if ( email.isEmpty() )return false;
    if (email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$"))return true;

    return false;
}
```

Clean Code Principles

- Meaningful names
- Small Functions
- Boy Scout Rule
- Avoid Magic number/string
- Consistent Formatting
- RIP (Return If Possible)
- **YAGNI (You Aren't Gonna Need It):**
  - **Do not implement functionality until it is actually necessary**
  - **Remove codes that are not in use.**

Clean Code Principles

- ➔ Meaningful names
- ➔ Small Functions
- ➔ Boy Scout Rule
- ➔ Avoid Magic number/string
- ➔ Consistent Formatting
- ➔ YAGNI
- ➔ **KISS (Keep It Simple, Stupid): If a solution is complicated, always ask if there is a more straightforward way.**

Clean Code Principles

- ➔ Meaningful names
- ➔ Small Functions
- ➔ Boy Scout Rule
- ➔ Avoid Magic number/string
- ➔ Consistent Formatting
- ➔ YAGNI
- ➔ KISS
- ➔ **DRY(Don't Repeat Yourself):** **Avoids code duplication**

Clean Code Principles

- Meaningful names
- Small Functions
- Boy Scout Rule
- Avoid Magic number/string
- Consistent Formatting
- YAGNI
- KISS
- DRY
- **SOLID**
  - **S** - Single Responsibility Principle (SRP)
  - **O** - Open/Closed Principle (OCP)
  - **L** - Liskov Substitution Principle (LSP)
  - **I** - Interface Segregation Principle (ISP)
  - **D** - Dependency Inversion Principle (DIP)

S - Single Responsibility Principle (SRP)

```
public void postDebit(String accountId, BigDecimal amount) {

    if (amount.signum() <= 0) {  throw new IllegalArgumentException("Invalid amount"); }

    // Business logic
    BigDecimal fee = amount.multiply(new BigDecimal("0.01"));
    BigDecimal total = amount.add(fee);

    // Persistence
    jdbcTemplate.update("UPDATE account SET balance = balance - ? WHERE id = ?",
total, accountId);

    // Notification
    String message = "Account: " + accountId + ", Amount: " + total;
    emailService.send( "ops@bank.com", "Debit posted", message);

}
```

S - Single Responsibility Principle (SRP)

```
public void postDebit(String accountId, BigDecimal amount)
{
    BigDecimal total = calculateTotalDebit(amount);
    updateBalance(accountId, total);
    notifyDebit(accountId, total);
}
```

O - Open/Closed Principle (OCP)

```
public BigDecimal calculateFee(String channel,
                               BigDecimal amount) {

    if ("ATM".equals(channel)) {
        return amount.multiply(new BigDecimal("0.015"));
    }

    else if ("BKASH".equals(channel)) {
        return new BigDecimal("50");
    }

    return BigDecimal.ZERO;
}
```

O - Open/Closed Principle (OCP)

```
public interface FeePolicy {

    boolean supports(String channel);
    BigDecimal calculate(BigDecimal amount);
}

public class AtmFeePolicy implements FeePolicy {

    public boolean supports(String channel) {
        return "ATM".equals(channel);
    }

    public BigDecimal calculate(BigDecimal amount) {
        return amount.multiply(new BigDecimal("0.015"));
    }
}
```

O - Open/Closed Principle (OCP)

```
public class BkashFeePolicy implements FeePolicy {

    public boolean supports(String channel) {

        return "BKASH".equals(channel);
    }

    public BigDecimal calculate(BigDecimal amount) {
        return new BigDecimal("50");
    }
}

public BigDecimal calculateFee(String channel, BigDecimal amount) {

    for (FeePolicy policy : policies) {

        if (policy.supports(channel)) {
            return policy.calculate(amount); }
    }
    return BigDecimal.ZERO;
}
```

L - Liskov Substitution Principle (LSP)

```
class Account {
    protected BigDecimal balance = BigDecimal.ZERO;
    // Contract: debit must be positive and reduces balance
    public void debit(BigDecimal amount) {
        if(amount.signum() <= 0) throw new IllegalArgumentException("amount > 0");
        if(balance.compareTo(amount) < 0) throw new IllegalStateException("Insufficient funds");
        balance = balance.subtract(amount);
    }
}

class OverdraftAccount extends Account {
    @Override
    public void debit(BigDecimal amount) {
        // "Improvement": allow negative to mean credit
        if(amount.signum() < 0) {
            balance = balance.add(amount.abs()); // surprise behavior
            return;
        }
        // "Improvement": no insufficient-funds rule
        balance = balance.subtract(amount);
    }
}
```

L - Liskov Substitution Principle (LSP)

What breaks (LSP violation)

The subtype changes the base contract:

- Base Account.debit(x) rejects non-positive amounts; subtype silently accepts negatives.
- Base enforces insufficient funds; subtype removes it.
- Code that works with Account will behave unexpectedly with OverdraftAccount.

If a subtype cannot be used anywhere the base type is expected without changing correctness, LSP is violated.

L - Liskov Substitution Principle (LSP)

```
class Account {

    private BigDecimal balance = BigDecimal.ZERO;
    private final DebitPolicy policy;

    Account(DebitPolicy policy) { this.policy = policy; }

    // Contract remains stable
    public void debit(BigDecimal amount) {

        if (amount.signum() <= 0) {
             throw new IllegalArgumentException("amount > 0");
        }
        policy.validate(balance, amount);
        balance = balance.subtract(amount);
    }
}
```

L - Liskov Substitution Principle (LSP)

```
interface DebitPolicy {
    void validate(BigDecimal balance, BigDecimal amount);
}

class NoOverdraftPolicy implements DebitPolicy {
    public void validate(BigDecimal balance, BigDecimal amount) {
        if(balance.compareTo(amount) < 0)
            throw new IllegalStateException("Insufficient funds");
    }
}

class LimitedOverdraftPolicy implements DebitPolicy {
    private final BigDecimal overdraftLimit; // e.g. 5000
    LimitedOverdraftPolicy(BigDecimal overdraftLimit)
    { this.overdraftLimit = overdraftLimit; }

    public void validate(BigDecimal balance, BigDecimal amount) {
        BigDecimal after = balance.subtract(amount);
        if(after.compareTo(overdraftLimit.negate()) <0)
          throw new IllegalStateException("Overdraft limit exceeded");
    }
}
```

I - Interface Segregation Principle (ISP)

Clients should not be forced to depend on methods they do not use

```
interface BankingService {
    void deposit(double amount);
    void withdraw(double amount);
    void generateMonthlyStatement();
    void runEndOfDayBatch();
}

class ATMService implements BankingService {

    public void deposit(double amount) { System.out.println("Depositing " + amount); }
    public void withdraw(double amount) { System.out.println("Withdrawing " + amount); }
    public void generateMonthlyStatement() { throw new UnsupportedOperationException(); }
    public void runEndOfDayBatch() { throw new UnsupportedOperationException(); }
}
```

I - Interface Segregation Principle (ISP)

```
interface AccountOperations {
    void deposit(double amount);
    void withdraw(double amount);
}

interface StatementService {
    void generateMonthlyStatement();
}

interface BatchService {
    void runEndOfDayBatch();
}
```

I - Interface Segregation Principle (ISP)

```
class ATMService implements AccountOperations {

    public void deposit(double amount) {
        System.out.println("Depositing " + amount);
    }

    public void withdraw(double amount) {
        System.out.println("Withdrawing " + amount);
    }
}
```

D - Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules.

Both should depend on abstractions.

```
class EmailService {

    public void send(String message) {
        System.out.println("Sending email: " + message);
    }
}

class PaymentService {

    private EmailService emailService = new EmailService();
    public void processPayment() {
        System.out.println("Processing payment");
        emailService.send("Payment completed");
    }
}
```

D - Dependency Inversion Principle (DIP)

Problem

- PaymentService depends directly on EmailService.
- If we switch to SMS or Kafka → modify PaymentService.
- Business logic tied to delivery mechanism.

D - Dependency Inversion Principle (DIP)

```
interface NotificationService {
    void send(String message);
}

class EmailService implements NotificationService {
    public void send(String message) {
        System.out.println("Sending email: " + message);
    }
}

class PaymentService {
    private NotificationService notificationService;
    public PaymentService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    public void processPayment() {
        System.out.println("Processing payment");
        notificationService.send("Payment completed");
    }
}
```

Thank you